

Module 2: Methods, Collections & Exceptions

Beginner · High Priority

1. Methods

- **Definition:** A method is a reusable block of code that performs a task.

```
public static int add(int a, int b) {  
    return a + b;  
}
```

- **Parameter Passing:** Java is always **pass-by-value**.
 - For primitives (int, double, etc.) — a copy of the value is passed.
 - For objects — a copy of the *reference* is passed, so the method can modify the object's internal state, but reassigning the reference inside the method does not affect the caller's variable.
- **Method Overloading:** Multiple methods with the same name but different parameter lists (different number/type of arguments).

```
void print(int a) { }  
void print(String a) { }  
void print(int a, int b) { }
```

- **Scope Rules:**
 - Variables declared inside a method are **local** — only visible within that method.
 - Variables declared at class level (fields) are visible to all methods in the class.
 - Block scope: variables inside {} (e.g., in a loop) only exist within that block.

2. Java Collections Framework

Collections store and manage groups of objects. Key interfaces: List, Set, Map.

ArrayList (List)

- Resizable array, allows duplicates, maintains insertion order.

```
ArrayList<String> names = new ArrayList<>();  
names.add("Alice");  
names.add("Bob");  
names.remove("Alice");  
names.get(0);  
names.size();
```

HashMap (Map)

- Stores key-value pairs. No duplicate keys. No guaranteed order.

```
HashMap<String, Integer> ages = new HashMap<>();
```

```
ages.put("Alice", 30);
ages.get("Alice");
ages.containsKey("Bob");
ages.remove("Alice");
```

HashSet (Set)

- Stores unique elements only, no duplicates, no guaranteed order.

```
HashSet<String> emails = new HashSet<>();
emails.add("a@x.com");
emails.add("a@x.com"); // ignored, duplicate
```

Collection	Duplicates?	Ordered?	Key-Value?
ArrayList	Yes	Insertion order	No
HashSet	No	No	No
HashMap	Keys: No	No	Yes

3. Exception Handling

- **try-catch**: Handles runtime errors gracefully instead of crashing the program.

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("This always runs.");
}
```

- **throw**: Manually throw an exception.

```
if (age < 0) {
    throw new IllegalArgumentException("Age cannot be negative");
}
```

- **throws**: Declares that a method might throw a checked exception, passing responsibility to the caller.

```
void readFile() throws IOException { ... }
```

- **Common Exceptions**: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, NumberFormatException, IOException.

4. Basic File Reading/Writing

```
import java.io.*;
import java.util.Scanner;

// Writing to a file
FileWriter writer = new FileWriter("data.txt");
writer.write("Hello, file!");
writer.close();

// Reading from a file
File file = new File("data.txt");
Scanner reader = new Scanner(file);
while (reader.hasNextLine()) {
    System.out.println(reader.nextLine());
}
```

```
reader.close();
```

5. Project: Student Management System

A console app demonstrating everything above — see `StudentManagementSystem.java`: - Uses an `ArrayList<Student>` to store records and a `HashMap<Integer, Student>` for fast ID lookup. - Custom methods for add, edit, delete, and search (overloaded search by ID and by name). - Custom exception (`StudentNotFoundException`) with try-catch handling around invalid input. - Saves and loads records to/from a text file.

Quick Recap

Topic	Key Takeaway
Methods	Reusable code blocks; pass-by-value; overloading by signature
Collections	<code>ArrayList</code> (ordered, duplicates), <code>HashSet</code> (unique), <code>HashMap</code> (key-value)
Exceptions	try-catch-finally to handle errors; throw/throws for custom & checked exceptions
File I/O	<code>FileWriter</code> to write, <code>Scanner/File</code> to read